Quick Commerce Backend — Developer Onboarding Guide

This document explains how our backend is architected, how the project is structured, and everything that needs to be configured to run and ship the service. Read this before touching code.

---

1. Architectural Approach

We are building a **modular monolith**, not microservices — at least to start.

Why: at this stage we don't yet know where the real service boundaries are, and the operational overhead of microservices (service discovery, distributed tracing, network latency between order → payment → delivery) slows us down before we have real scale problems.

A modular monolith means: one deployable Node app, internally split into clean, independent modules (own routes, services, data access). If a module — say Delivery — needs to become its own service later, we extract it without a rewrite, because module boundaries are already respected in code.

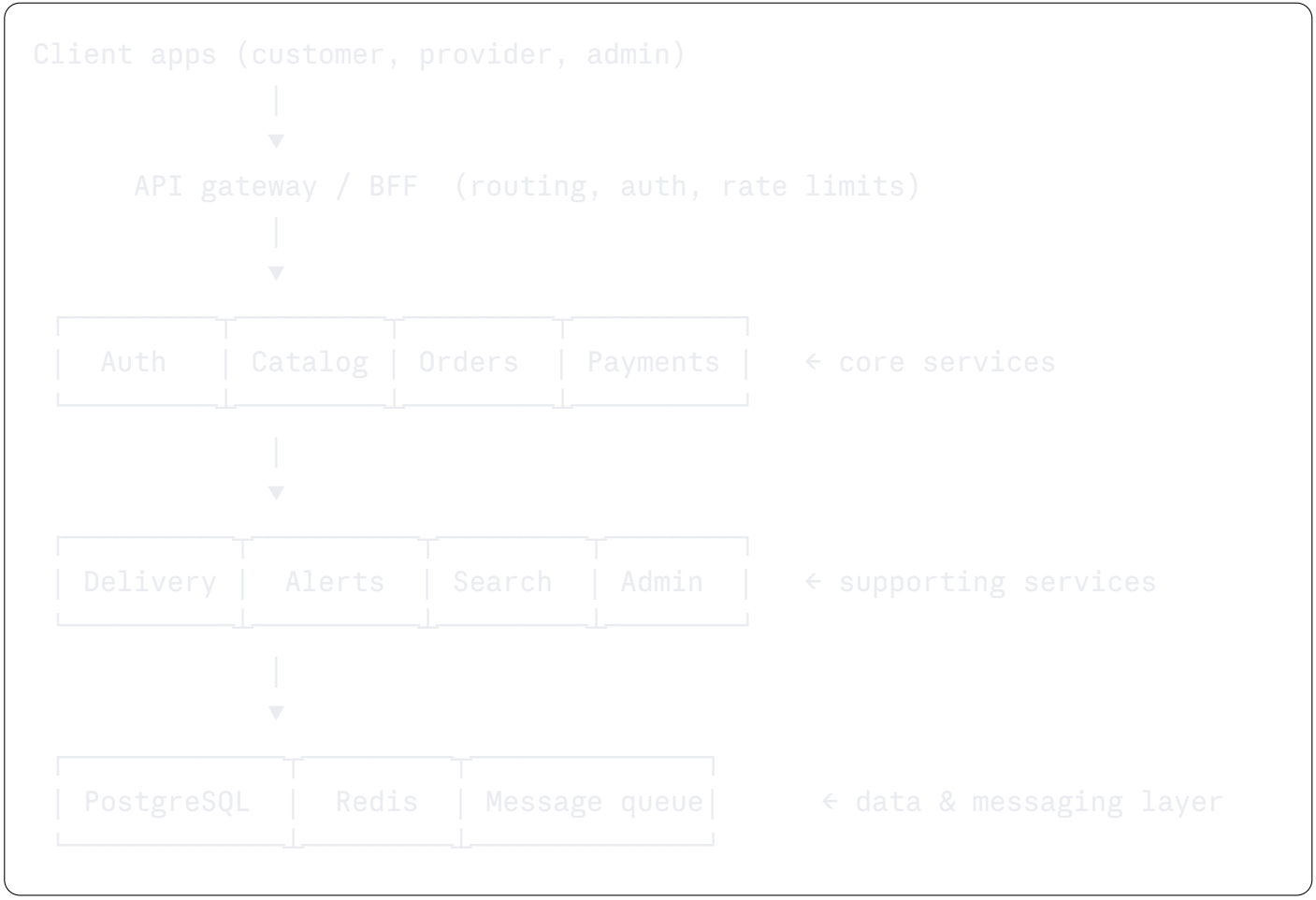
Rule: a controller never talks to the DB directly. A module's service never reaches into another module's repository. Cross-module communication goes through the other module's service, or through an internal event (e.g. Orders emits `order.placed`, Delivery listens for it).

---

2. Tech Stack

Layer	Choice	Why
Runtime	Node.js + TypeScript	Compile-time safety for order/payment logic
Framework	NestJS or Express/Fastify + conventions	Structure & DI if Nest; lighter weight otherwise
Primary DB	PostgreSQL	ACID transactions for orders, payments, inventory
ORM	Prisma or Drizzle	Type-safe queries, migrations
Cache / sessions / geo	Redis	Also used for `GEOADD`/`GEOSEARCH` nearest-rider lookups
Search (later)	Elasticsearch / Meilisearch	Once catalog search outgrows SQL `LIKE`
Queue	BullMQ (Redis-backed) → Kafka/RabbitMQ later	Async jobs, order timeouts, payout batches
Realtime	Socket.io / WS gateway	Live order & rider tracking
Payments	Razorpay / Stripe / PayU	Never build settlement logic ourselves
Infra	Docker (local + prod)	Consistent environments, easy path to ECS/K8s

3. High-Level Architecture



---

4. Project Structure

Feature-first, modular monolith layout:

```
src/
├── modules/
│   ├── auth/
│   │   ├── auth.controller.ts
│   │   ├── auth.service.ts
│   │   ├── auth.repository.ts
│   │   ├── auth.routes.ts
│   │   ├── dto/
│   │   └── auth.types.ts
│   ├── catalog/
│   ├── orders/
│   │   ├── order.state-machine.ts   ← explicit state machine, not
scattered ifs
│   │   └── ...
│   ├── payments/
│   ├── delivery/                    ← provider assignment, live location,
geo queries
│   ├── notifications/
│   └── users/
├── common/
│   ├── middlewares/                 (auth guard, rate limiter, error
handler)
│   ├── errors/                      (typed AppError classes)
│   ├── utils/
│   └── validators/
├── config/                          (env schema validation — zod/joi,
fail fast on missing env)
├── jobs/                            (BullMQ processors: order-timeout,
payout-batch, notification-retry)
├── events/                          (internal event bus / pub-sub
definitions)
├── db/
│   ├── migrations/
│   └── seed/
├── sockets/                         (real-time tracking gateway)
├── app.ts
└── server.ts
```

Module template every developer follows: `controller → service → repository`. Business logic lives in the service. The repository is the only thing that touches the DB.

---

5. Core Design Decisions (read before writing business logic)

- **Order state machine, explicitly.** Order status is never a loose string with scattered `if` checks. Allowed transitions (`placed → confirmed → assigned → picked_up → delivered / cancelled`) live in one file, and invalid transitions are rejected

there.

- **Idempotency on money and order-mutating endpoints.** Payment webhooks and app clients retry. Every "create order", "capture payment", "confirm delivery" endpoint requires an idempotency key.
- **Inventory concurrency.** Decide up front: row-level locking (`SELECT ... FOR UPDATE`), atomic `UPDATE ... WHERE stock > 0`, or Redis-based reservation with TTL. Do not defer this decision.
- **Geo-matching strategy.** Redis GEO commands to start; PostGIS if we need delivery-zone polygons or ETA-aware assignment later.
- **Async by default** for anything off the critical path — notifications, analytics events, payout batching, review aggregation all go through the queue.
- **Config validated at boot**, not discovered at runtime.
- **API versioning** (`/api/v1/...`) from day one — mobile clients in the wild can't be force-updated.
- **Rate limiting** specifically on auth, OTP, and order-creation endpoints — the most abused surfaces in commerce apps.

---

6. Full Configuration Checklist

6.1 Environment & config

- ▢ `.env.example` committed (never the real `.env`)
- ▢ Env validation on boot (zod/joi) — crash immediately on missing vars
- ▢ Separate configs per environment: `local`, `staging`, `production`
- ▢ Secrets manager for staging/prod (AWS Secrets Manager / GCP Secret Manager / Doppler)

6.2 Docker

- ▢ `Dockerfile` — multi-stage build (builder compiles TS, runner copies `dist/` + prod deps only)
- ▢ `.dockerignore` (node_modules, .git, .env, dist)
- ▢ `docker-compose.yml` for local dev — app, Postgres, Redis, Bull Board, pgAdmin/Adminer
- ▢ Healthcheck defined (`/health` endpoint) in Dockerfile/compose

6.3 Database

- ▢ Postgres — local via Docker; managed (RDS/Cloud SQL/Supabase/Neon) for staging+prod
- ▢ ORM schema + migrations folder (Prisma/Drizzle)
- ▢ Migration workflow per env decided (auto-on-deploy vs manual approval gate for prod)
- ▢ Seed script for local dev

- [ ] Connection pooling (PgBouncer or ORM's built-in pool), sized for instance count
- [ ] Automated daily backups + point-in-time recovery
- [ ] Read-replica pattern decided (reads via repository layer, even if unused today)

6.4 Redis

- [ ] Logical separation documented: cache vs session vs geo vs queue (separate instances/DB indexes/key prefixes)
- [ ] Eviction policy set (`allkeys-lru` for pure cache instance)
- [ ] TTLs defined per key type

6.5 Queue / async jobs

- [ ] BullMQ (or Kafka/RabbitMQ) connected to its own Redis/broker
- [ ] Bull Board (or equivalent) dashboard for job visibility
- [ ] Retry policy + dead-letter handling per job type

6.6 CI (on every PR)

- [ ] Lint (ESLint) + format check (Prettier)
- [ ] Type-check (`tsc --noEmit`)
- [ ] Unit + integration tests (Postgres/Redis as CI service containers)
- [ ] Docker build check
- [ ] Dependency audit (`npm audit` / Snyk / Dependabot)
- [ ] Branch protection — PRs blocked unless CI passes

6.7 CD (on merge/tag)

- [ ] Build & push Docker image to registry, tagged with git SHA
- [ ] Auto-deploy to staging on merge to `main`/`develop`
- [ ] Manual approval gate for production
- [ ] DB migrations run as a deploy step, before new containers take traffic
- [ ] Rollback strategy defined (tagged images or blue-green/canary)

6.8 Hosting / orchestration

- [ ] Platform chosen: ECS Fargate / GKE / EKS / Railway / Render / single VM for MVP
- [ ] Load balancer + auto-scaling rules (CPU or queue depth, not just request count)
- [ ] Separate scaling group for API vs background workers

6.9 Observability

- [ ] Structured logging (pino) shipped to CloudWatch/Datadog/Better Stack
- [ ] Error tracking (Sentry) in both API and worker processes
- [ ] APM/tracing (Datadog, New Relic, or OpenTelemetry + Jaeger) if budget allows
- [ ] Uptime monitoring + alerting on `/health`
- [ ] Dashboards for: order success rate, payment failure rate, avg. assignment time, queue lag

6.10 Security

- ☐ Helmet + explicit CORS config (never `*`)
- ☐ Rate limiting per-IP and per-user on auth, OTP, order-create
- ☐ JWT rotation / refresh-token strategy decided
- ☐ Input validation on every endpoint (class-validator/zod)
- ☐ Dependency scanning in CI (Snyk/Dependabot)
- ☐ HTTPS everywhere, HSTS enabled

6.11 Third-party integrations

- ☐ Payment gateway sandbox account + webhook endpoint registered
- ☐ SMS/OTP provider (Twilio/MSG91)
- ☐ Push notifications (FCM/APNs or OneSignal)
- ☐ Maps/geocoding provider (Google Maps/Mapbox)
- ☐ Cloud storage (S3/GCS) for product photos & provider documents

6.12 Repo hygiene

- ☐ Branching strategy documented (trunk-based or git-flow)
- ☐ PR template + commit convention (Conventional Commits recommended)
- ☐ README with local setup steps (`docker-compose up`, seed command, test command)

---

7. Getting Started (for a new developer)

1. Clone the repo, copy `.env.example` to `.env`, fill in local values.
2. Run `docker-compose up` — this brings up the app, Postgres, and Redis together.
3. Run migrations: `<migration command>` (see `db/migrations`).
4. Run the seed script to get sample users, catalog items, and test orders.
5. Run `npm run dev` (or the compose service handles this) and hit `/health` to confirm everything is wired up.
6. Run the test suite before making any changes, to confirm the baseline is green.
7. Read `src/modules/orders/order.state-machine.ts` first — it's the most important file in the codebase to understand before touching order-related logic.

---

8. Open Questions / To Be Decided

Use this section to track decisions the team still needs to make (fill in as you go):

- ☐ Managed Postgres provider for staging/prod
- ☐ Hosting platform (ECS vs GKE vs Railway/Render vs VM)
- ☐ Payment gateway
- ☐ SMS/OTP and push notification providers
- ☐ Branching strategy